Videogame Review API – Documentation

Contents

POST `/token/` 1

POST `/token/refresh/` 2

POST `/users/` 3

GET `/users/{user_id}` 4

GET `/users/{user_id}/reviews/` 7

GET `/games/` 8

GET `/games/{game_id}` 13

GET `/games/{game_id}/reviews/` 15

GET `/games/{game_id}/analytics` 16

POST `/reviews/` 18

GET `/reviews/{review_id}` 19

PATCH `/reviews/{review_id}` 21

DELETE `/reviews/{review_id}` 22

POST `/token/`

Description

Returns **access** and **refresh** JSON Web Tokens in the response body, using the provided **username** and **password** in the request body. If successful, returns a HTTP `200 OK`.

If the **username** or **password** is incorrect, returns a HTTP `401 Unauthorized`.

If the **username** or **password** is missing, returns a HTTP `400 Bad Request`.

The **access** token is needed to perform the following API endpoints:

- **POST** `/reviews/`
- **PATCH** `/reviews/{review_id}`
- **DELETE** `/reviews/{review_id}`

Request Structure

Header

Content-Type: application/json

Body

username: string – username of user trying to authenticate

password: string – corresponding password of the username being sent

Example

```
{
      "username": "user_being_authenticated",
      "password": "this_is_a_password"
}
```

Response Structure

Header

Content-Type: application/json

Body

access: string – a JSON Web Token that can be used to authenticate a user's identity without a **username** and **password**. Valid for 5 minutes.

refresh: string – a JSON Web Token that can be used to provide a new **access** token via **POST** `/token/refresh/`. Valid for 1 day.

Example

```
{
      "access": "access_token"
      "refresh": "refresh_token",
}
```

POST `/token/refresh/`

Description

Returns an **access** JSON Web Token in the response body, using the provided refresh JSON Web Token in the request body. If successful, returns a HTTP `200 OK`.

If the **refresh** token is invalid, returns a HTTP `401 Unauthorized`.

If the **refresh** token is missing, returns a HTTP `400 Bad Request`.

The **access** token is needed to perform the following API endpoints:

- **POST** `/reviews/`
- **PATCH** `/reviews/{review_id}`
- **DELETE** `/reviews/{review_id}`

Request Structure

Header

Content-Type: application/json

Body

refresh: string – a valid JSON Web Token for the user trying to get a new **access** token.

Example

```
{
        "refresh": "refresh_token","
}
```

Response Structure

Header

Content-Type: application/json

Body

access: string – a JSON Web Token that can be used to authenticate a user's identity without a **username** and **password**. Valid for 5 minutes.

Example

```
{
       "access": "new_access_token",
}
```

POST `/users/`

Description

Creates a new user using the provided **username** and **password** in the request body. If successful, returns a HTTP `201 Created`.

If the **username** is already taken, returns a HTTP `409 Conflict`.

If the **password** is not valid, returns a HTTP `400 Bad Request`.

If the request body has missing or misnamed keys, returns a HTTP `400 Bad Request`.

Passwords are encrypted using a PBKDF2 algorithm with a SHA256 hash before storage.

Request Structure

Header

Content-Type: application/json

Body

username: string – unique username for the user being created. Must be at least 3 characters long.

password: string – must be at least 12 characters long, non-numeric, and not too similar to the **username**.

Example

```
{
      "username": "new_user",
      "password": "this_is_a_password"
}
```

Response Structure

Header

Content-Type: application/json

Body

username: string – newly created user's username (matching **username** in request body).

url: link – a retrieves new user's profile via **GET** `/users/{user_id}`

Example

```
{
      "username": "new_user",
      "url": "https://hoeyinfoong.pythonanywhere.com/api/users/12345"
}
```

GET `/users/{user_id}`

Description

Returns a user profile matching the **user_id** path variable with statistics based on the user's reviews, and a HTTP `200 OK`.

If the **user_id** is invalid or missing, returns a HTTP `404 Not Found`.

Request Structure

Path Variable

user_id: integer – the user id for the user being retrieved.

Response Structure

Header

Content-Type: application/json

Body

id: integer – user id matching **user_id** path variable.

username: string – the user's username.

url: link – for this resource.

reviews: link – retrieves the user's reviews via **GET** `/users/{user_id}/reviews/`

reviews_n: integer – number of reviews that the user has authored.

average_score: integer – the user's average review score out of 100, rounded down to nearest integer.

reviews_hist: list – user's review score distribution, grouping scores in intervals of 20. For example [2, 5, 10, 60, 20] means there are:

- Two reviews with a score 0-20
- Five reviews with a score 21-40
- Ten reviews with a score 41-60
- Sixty reviews with a score 61-80
- Twenty reviews with a score 81-100.

favorite_genres: list – the user's three favorite genres, determined and ordered by their average score for games in each genre (for genres with at least 1 review).

favorite_games: list – the user's three best games, determined and ordered by highest review score:

> **url:** link – retrieves the review for the game via **GET** `/reviews/{review_id}`
>
> **game:** representation of the game:
>
> > **title:** string – game title.
> >
> > **url:** link – retrieves the game via **GET** `/games/{game_id}`
>
> **score:** integer – review score out of 100.

Example

GET /users/3137


```
{
    "id": 3137,
    "username": "Broyax",
    "url": "https://hoeyinfoong.pythonanywhere.com/api/users/3137",
    "reviews": "https://hoeyinfoong.pythonanywhere.com/api/users/3137/reviews/",
    "reviews_n": 53,
    "average_score": 36,
    "reviews_hist": [
        21,
        11,
        9,
        9,
        3
    ],
    "favorite_genres": [
        "Western RPG",
        "Action RPG",
        "FPS"
    ],
    "favorite_games": [
        {
            "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/42644",
            "game": {
                "title": "Mass Effect 2",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/24"
            },
            "score": 100
        },
        {
            "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/108394",
            "game": {
                "title": "Star Wars: Knights of the Old Republic",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/85"
            },
            "score": 100
        },
        {
            "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/129058",
            "game": {
                "title": "Mass Effect 3",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/117"
            },
            "score": 90
        }
    ]
}
```

GET `/users/{user_id}/reviews/`

Description

Returns a list of reviews by the user specified by the **user_id** path variable, ordered by the most recent publish date. If successful, returns a HTTP `200 OK`.

If the **user_id** is empty or invalid, returns a HTTP `404 Not Found`.

There is a limit of 25 reviews per page, and additional reviews can be found using the **page** query parameter e.g. **GET** `/users/{user_id}/reviews/?page=2`. An invalid **page** will return a HTTP `404 Not Found`.

Request Structure

Path Variable

user_id: integer – the user id for the user whose reviews are being retrieved.

Query Parameters

page: integer – the page number for viewing the list of reviews. 25 reviews per page.

Response Structure

Header

Content-Type: application/json

Body

List of reviews: see the request body structure from **GET** `/reviews/{review_id}` for more details on the structure of an individual review.

Example

GET `/users/3137/reviews/`: Only the first two reviews are shown as an example.

```
[
    {
        "id": 95325,
        "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/95325",
        "user": {
            "username": "Broyax",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/users/3137"
        },
        "game": {
            "title": "Metaphor: ReFantazio",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/67"
        },
        "date": "2024-12-14",
        "score": 0,
        "content": "Sous ce titre un peu étrange et ronflant… un fiasco !"
    },
    {
        "id": 87578,
        "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/87578",
        "user": {
            "username": "Broyax",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/users/3137"
        },
        "game": {
            "title": "God of War",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/52"
        },
        "date": "2024-11-17",
        "score": 0,
        "content": "Déjà le titre (comme beaucoup d'autres… de merde !"
    },
    …
]
```


GET `/games/`

Description

Returns an analysis of and list of all games in the database, and a HTTP `200 OK`. Can return a HTTP `404 Not Found` if the **page** query parameter (see the rest of this section) is invalid.

The list of games is paginated to display 25 games per page. For example, to view games 26-50 in the second page, use **GET** `/games/?page=2`

Other query parameters can be specified to filter to list of games retrieved. The analysis will be based on games filtered by all query parameters excluding page, e.g. **GET** `/games/?genre=action&platform=pc&page=2` contains the analysis of all action games on the PC platform.

The analysis for each page will remain the same; the only change is the list of games between pages.

Invalid query parameter keys return a HTTP `400 Bad Request`, but valid keys with bad values will just return an empty list (no games found with those filters) and a HTTP `200 OK`.

Request Structure

Query Parameters

page: integer – page number for the list of games

title: string – filters games by title (case-insensitive search for titles that contain this value)

genre: string – filters games by genre (case-insensitive exact match)

platform: string – filters games by platform (case-insensitive exact match)

developer: string – filters games by developer (case-insensitive exact match)

publisher: string – filters games by publisher (case-insensitive exact match)

Response Structure

Header

Content-Type: application/json

Body

analytics: finds the top-rated and trending games for the list of games retrieved:

- **top_rated:** list – the top 5 games, determined and ordered by highest average score.
- **game:** representation of a game:
  - **title:** string – game title.
  - **url:** link – retrieves the game via **GET** `/games/{game_id}`
  - **average_score:** integer – average score of the game out of 100, rounded down to the nearest integer.
- **trending:** list – the most trending 5 games, determined and ordered by the number of reviews from the most recent 100 reviews for this list of games:
  - **title:** string – game title.
  - **url:** link – retrieves the game via **GET** `/games/{game_id}`

games: list – games retrieved by the request, sorted by most recent release date. Limit of 25 per page:

- **title:** string – game title.
- **url:** link – retrieves the specific game via **GET** `/games/{game_id}`

release_date: string – release date of the game in YYYY-MM-DD format.

Example without query parameters (all games)

GET `/games/`: Only the first three games for **top_rated** and **trending** are shown, while only the first two for **games** are shown.


```
{
    "analytics": {
        "top_rated": [
            {
                "game": {
                    "title": "Disco Elysium: The Final Cut",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/495"
                },
                "average_score": 100
            },
            {
                "game": {
                    "title": "The Last of Us Part II",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/91"
                },
                "average_score": 99
            },
            {
                "game": {
                    "title": "NFL 2K1",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/15"
                },
                "average_score": 98
            },
            …
        ],
        "trending": [
            {
                "title": "Astro Bot",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/55"
            },
            {
                "title": "The Legend of Zelda: Tears of the Kingdom",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/29"
            },
            {
                "title": "Elden Ring",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/26"
            },
            …
        ]
    },
    "games": [
        {
            "title": "WWE 2K25",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/2465",
            "release_date": "2025-03-14"
        },
        {
            "title": "Expelled!",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/1665",
            "release_date": "2025-03-12"
        },
        …
]
```

Example with page query parameter

GET `/games/?page=2`: Only the first three games for **top_rated** and **trending** are shown, while only the first two for **games** are shown. Note that analysis is the same as the previous example, but the list of games has changed.


```
{
    "analytics": {
        "top_rated": [
            {
                "game": {
                    "title": "Disco Elysium: The Final Cut",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/495"
                },
                "average_score": 100
            },
            {
                "game": {
                    "title": "The Last of Us Part II",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/91"
                },
                "average_score": 99
            },
            {
                "game": {
                    "title": "NFL 2K1",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/15"
                },
                "average_score": 98
            },
            …
        ],
        "trending": [
            {
                "title": "Astro Bot",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/55"
            },
            {
                "title": "The Legend of Zelda: Tears of the Kingdom",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/29"
            },
            {
                "title": "Elden Ring",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/26"
            },
            …
        ]
    },
    "games": [
        {
            "title": "MACROSS -Shooting Insight-",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/9731",
            "release_date": "2025-02-07"
        },
        {
            "title": "Keep Driving",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/1549",
            "release_date": "2025-02-06"
        },
        …
]
```

Example with filtering by query parameters

GET `/games/?genre=fps&platform=pc`: Only the first three games for **top_rated** and **trending** are shown, while only the first two for **games** are shown.


```
{
    "analytics": {
        "top_rated": [
            {
                "game": {
                    "title": "Portal 2",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
                },
                "average_score": 93
            },
            {
                "game": {
                    "title": " Half-Life: Alyx",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/96"
                },
                "average_score": 92
            },
            {
                "game": {
                    "title": " Half-Life 2",
                    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/98"
                },
                "average_score": 91
            },
            …
        ],
        "trending": [
            {
                "title": " BioShock Infinite",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/69"
            },
            {
                "title": "Portal 2",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
            },
            {
                "title": "BioShock",
                "url": "https://hoeyinfoong.pythonanywhere.com/api/games/17"
            },
            …
        ]
    },
    "games": [
        {
            "title": " Phantom Fury",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/9495",
            "release_date": "2024-04-23"
        },
        {
            "title": " Terminator: Resistance - Annihilation Line",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/9672",
            "release_date": "2021-12-10"
        },
        …
]
```

GET `/games/{game_id}`

Description

Returns the details of the game resource specified by the **game_id** path variable, and a HTTP `200 OK`.

If **game_id** is invalid, returns a HTTP `404 Not Found`.

If **game_id** is missing, uses the **GET** `/games/` endpoint instead.

Request Structure

Path Variable

game_id: integer – the id of the game being retrieved.

Response Structure

Header

Content-Type: application/json

Body

id: integer – game id matching **game_id** path variable.

title: string – game title.

url: link – for this resource.

release_date: string – release date of the game in YYYY-MM-DD format.

rating: string – maturity rating of the game.

description: string – game description.

genre: string – game genre.

platforms: list – platforms this game can be played on.

developers: list – developers that made the game.

publishers: list – publishers that published the game.

reviews: link – retrieves the game's reviews via **GET** `/games/{game_id}/reviews`

analytics: link – retrieve the game's analytics via **GET** `/games/{game_id}/analytics`

Example

```
GET /games/50
```

```
{
    "id": 50,
    "title": "Portal 2",
    "url": "https://hoeyinfoong.pythonanywhere.com/api/games/50",
    "release_date": "2011-04-19",
    "rating": "E10+",
    "description": "Valve is working on a full-length sequel to its award-
winning, mind-bending puzzle action game for the Xbox 360 and PC.",
    "genre": "FPS",
    "platforms": [
        "Xbox 360",
        "PlayStation 3",
        "PC"
    ],
    "developers": [
        "Valve Software"
    ],
    "publishers": [
        "Electronic Arts",
        "Valve Software"
    ],
    "reviews": "https://hoeyinfoong.pythonanywhere.com/api/games/50/reviews/",
    "analytics": "https://hoeyinfoong.pythonanywhere.com/api/games/50/analytics"
}
```


GET `/games/{game_id}/reviews/`

Description

Returns a list of the game's reviews for the game specified by the **game_id** path variable, ordered by most recent publish date. If successful, returns a HTTP `200 OK`.

If **game_id** is empty or missing, returns a HTTP `404 Not Found`.

There is a limit of 25 reviews per page, and additional reviews can be found using the **page** query parameter e.g. **GET** `/games/{game_id}/reviews/?page=2`. An invalid **page** returns a HTTP `404 Not Found`.

Request Structure

Path Variable

game_id: integer – the id of the game whose reviews are being retrieved.

Response Structure

Header

Content-Type: application/json

Body

List of reviews: see the request body structure from **GET** `/reviews/{review_id}` for more details on the structure of an individual review.

Example

GET `/games/50/reviews/`: Only the first two are shown as an example.

```
[
    {
        "id": 80871,
        "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/80871",
        "user": {
            "username": "convretcetaste",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/users/21964"
        },
        "game": {
            "title": "Portal 2",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
        },
        "date": "2025-03-08",
        "score": 80,
        "content": "creative and innovative for its time…"
    },
    {
        "id": 80579,
        "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/80579",
        "user": {
            "username": "Sweetfr19",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/users/18199"
        },
        "game": {
            "title": "Portal 2",
            "url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
        },
        "date": "2025-03-04",
        "score": 100,
        "content": "Best Valve Game I ever Played. Good Story. Good Puzzle."
    },
    …
]
```

GET `/games/{game_id}/analytics`

Description

Returns a simple analysis of the game matching the **game_id** path variable, and a HTTP `200 OK`.

Analysis includes an average score, score distribution, and the month it was most popular.

If **game_id** is invalid or empty, returns a HTTP `404 Not Found`.

Request Structure

Path Variable

game_id: integer – the id of the game whose analytics are being retrieved.

Response Structure

Header

Content-Type: application/json

Body

game: representation of the game that has been analysed.

> **title:** string – game title.
>
> **url:** link – retrieves the game via **GET** `/games/{game_id}`

reviews_n: integer – number of reviews for the game.

average_score: integer – average review score out of 100, rounded down to nearest integer.

reviews_hist: list – review score distribution, grouping scores in intervals of 20. For example

[2, 5, 10, 60, 20] means there are:

- Two reviews with a score 0-20
- Five reviews with a score 21-40
- Ten reviews with a score 41-60
- Sixty reviews with a score 61-80
- Twenty reviews with a score 81-100.

best_month: representation of the month the game was most popular, determined by the month it received the most reviews.

> **date:** string – the month the game was most popular in YYYY-MM-DD format.
>
> **reviews_n:** integer – number of reviews published that month for the game.

Example

GET `/games/50/analytics`

```
{
    "game": {
        "title": "Portal 2",
        "url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
    },
    "reviews_n": 3046,
    "average_score": 93,
    "reviews_hist": [
        39,
        19,
        70,
        290,
        2628
    ],
    "best_month": {
        "date": "2011-04-19",
        "reviews_n": 451
    }
}
```


POST `/reviews/`

Description

Creates a new review using the provided request body data, and if successful returns a HTTP `201 Created`.

A valid **access** JSON Web Token must be provided in the request header. The token can be retrieved from **POST** `/token/` with a **username** and **password**, or from **POST** `/token/refresh/` if the user has a valid **refresh** JSON Web Token.

If the **access** JSON Web Token is invalid or missing, returns a HTTP `401 Unauthorized`.

If the authenticated user already has an existing review for the provided game, returns a HTTP `409 Conflict`. Note that while the existing database has some users with multiple reviews for the same game, new reviews must adhere to this constraint (one review per user-game pair).

If the **game** specified in the request body is invalid, returns a HTTP `404 Not Found`.

If the request body data is incorrect or missing any needed key-value pairs, returns a HTTP `400 Bad Request`.

Request Structure

Header

Authorization: Must be in the format "`Bearer access_token`", where the **access** token is "`access_token`".

Content-Type: application/json

Body

game: integer – id of the game being reviewed.

date: string – publish date of review in YYYY-MM-DD format. It can not be later than the day the request is received.

score: integer – score out of 100.

content: string – content of the review. Optional.

Example

```
{
    "game": 22,
    "date": "2022-02-22",
    "score": 22,
    "content": "this is a new review"
}
```

Response Structure

Header

Content-Type: application/json

Body

url: link – retrieves the newly created review via **GET** `/reviews/{review_id}`

Example

```
{
    "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/123456"
}
```

GET `/reviews/{review_id}`

Description

Returns the details of the review specified by the **review_id** path variable, and a HTTP `200 OK`

If **review_id** is invalid or missing, returns HTTP `404 Not Found`.

Request Structure

Path Variable

review_id: integer – id of the review being retrieved.

Response Structure

Header

Content-Type: application/json

Body

id: integer – review id matching the **review_id** path variable.

url: link – for this review.

user: representation of the author of the review.

- **username:** string - author's username.
- **url:** link - retrieves the author's profile via **GET** `/users/{user_id}`

game: representation of the game that the review is reviewing.

- **title:** string – game title.
- **url:** link – retrieves the game via **GET** `/games/{game_id}`

date: string – the review's publish date in YYYY-MM-DD format.

score: integer – review score out of 100.

content: string – content of the review.

Example

GET `/reviews/1`

```
{
    "id": 1,
    "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/1",
    "user": {
        "username": "Filipe69",
        "url": "https://hoeyinfoong.pythonanywhere.com/api/users/2"
    },
    "game": {
        "title": "The Legend of Zelda: Ocarina of Time",
        "url": "https://hoeyinfoong.pythonanywhere.com/api/games/1"
    },
    "date": "2023-01-30",
    "score": 100,
    "content": "C'est jeu est le meilleur jeu qui ai pu existé, une
histoire incroyable des boss et un game play parfait !"
},
```

PATCH `/reviews/{review_id}`

Description

Updates an existing review matching the **review_id** path variable, using the provided request body data. If successful it returns a HTTP `204 No Content`.

A valid **access** JSON Web Token must be provided in the request header. The token can be retrieved from **POST** `/token/` with a **username** and **password**, or from **POST** `/token/refresh/` if the user has a valid **refresh** JSON Web Token.

If the **access** JSON Web Token is invalid or missing, returns a HTTP `401 Unauthorized`.

If an authenticated user is trying to update a review that they did not author, returns a HTTP `403 Forbidden`.

If **review_id** is invalid, returns a HTTP `404 Not Found`.

If **review_id** is missing, returns a HTTP `400 Bad Request`.

If the request body data is incorrect or missing any needed key-value pairs, returns a HTTP `400 Bad Request`.

Request Structure

Header

Authorization: Must be in the format "`Bearer access_token`", where the **access** token is "`access_token`".

Content-Type: application/json

Body

id: integer – review id matching **review_id** path variable.

score: integer – new score out of 100. Optional if **content** was provided.

content: string – new content of the review. Optional if **score** was provided.

Example

```
{
    "id": 123456,
    "score": 100,
    "content": “Updating review content.”
}
```

DELETE `/reviews/{review_id}`

Description

Deletes an existing review matching the **review_id** path variable. If successful it returns a HTTP `204 No Content`.

A valid **access** JSON Web Token must be provided in the request header. The token can be retrieved from **POST** `/token/` with a **username** and **password**, or from **POST** `/token/refresh/` if the user has a valid **refresh** JSON Web Token.

If the **access** JSON Web Token is invalid or missing, returns a HTTP `401 Unauthorized`.

If an authenticated user is trying to delete a review that they did not author, returns a HTTP `403 Forbidden`.

If **review_id** is invalid, returns a HTTP `404 Not Found`.

If **review_id** is missing, returns a HTTP `400 Bad Request`.

Request Structure

Header

Authorization: Must be in the format “`Bearer access_token`”, where the **access** token is “`access_token`”.

Content-Type: application/json